

PRESENTATION SCRIPT · SMART INDIA HACKATHON

What to actually say while you present POLAR-EMS

A spoken script built around your real, working demo — not slides describing it. Every cue below points at something that genuinely happens when you click it.

FULL VERSION	FALLBACK	SEGMENTS	SPEAKERS
~6 minutes	30 seconds	7, independently trimmable	1 (splits to 2, see below)

How to use this

1 Read it out loud twice, verbatim

Then stop reading it and start saying it in your own words from memory. A memorized script sounds like a presentation; a read-aloud one sounds like a hostage note.

2 Time yourself once, for real

The 6-minute target assumes a measured, confident pace. If you're consistently running long, cut the Forecast half of Segment 4 first — it's the least essential beat.

3 Cues in the dashed boxes are for you, not the judges

They tell you what to click. Never read a cue out loud — if you catch yourself saying "now I will click the Optimizer tab," stop and just click it.

4 Know the 30-second version cold

If a judge says "you have one minute," you fall back to Section 2, not a panicked summary of everything.

30-second version (memorize this one first)

Your insurance policy if time gets cut without warning — which happens constantly in hackathon rounds.

"Polar research stations run on diesel shipped in at enormous cost, with internet that can vanish for weeks, and a power failure that's a safety emergency, not an inconvenience. POLAR-EMS is a proposed AI-driven energy management system that forecasts demand and renewable output, optimizes the diesel-battery-wind-solar mix in real time, and keeps making safe decisions with zero internet connection. [\[open the dashboard\]](#) And here it is, actually running — live, right now, computing every number you see."

Full script

Seven segments, each independently timed. Spoken text is in the light panels; anything in a dashed box is a stage direction for you.

1 · Hook & the problem

0:00 – 0:45

"Before I say anything about the technology, picture the situation. *[pause]* A research station at the bottom of the world. No power grid. Every litre of diesel arrives by ship, at a cost most of us would find absurd. The internet can disappear for weeks at a time. And if the power fails at minus 45 degrees, that's not an inconvenience — it's a safety emergency, for the people living there and the equipment keeping them alive.

That's the actual engineering problem we chose — not because it's dramatic, but because it forces you to get three things right at once: fuel efficiency, total autonomy, and safety that never depends on anything working perfectly."

2 · The solution, one sentence

0:45 – 1:15

"So we built POLAR-EMS — an AI-driven energy management system for polar research stations. In one sentence: it forecasts load and renewable generation ahead of time, continuously decides the optimal mix of battery, wind, solar and diesel, and does all of that at the edge, with a hard safety layer underneath that keeps working even if the AI

itself fails.

I want to show you this actually running — not slides about it."

CUE Open `polar-ems-web.vercel.app`, Dashboard tab, before you say the next line.

3 · Live dashboard

1:15 – 2:15

"This is the live dashboard — and I want to be upfront about something: this is a synthetic simulation, not real station telemetry, because none of us have access to an actual Antarctic station. But it isn't random numbers either — it's a physical model. Wind speed drives power through a real turbine power curve. Solar output follows the actual seasonal day-night cycle of a polar location, including true polar night. Every number on this screen — load, battery, diesel, renewables — is being computed live, right now, while I'm talking.

[point to the power flow diagram] Here's the power flow: wind and solar feed the load first, the battery covers the gap, and diesel only starts when it actually has to."

4 · Forecast + Optimizer (the core)

2:15 – 3:15

"Two things make this AI-driven rather than just a good-looking dashboard. First — forecasting. *[click Forecast tab]* It predicts load and renewable output ahead of time, and we score it honestly with MAE and RMSE against what actually happens — not a claimed accuracy, a measured one.

Second, and this is the core of the project — *[click Optimizer tab]* — a dispatch optimizer that runs two controllers side by side, on identical conditions: a simple rule-based baseline, and our forecast-aware optimizer. Every decision it makes comes with a plain-English reason. *[point to the reasoning box]* You can read exactly why it chose to discharge the battery instead of starting the generator. It is not a black box."

CUE If short on time, this segment survives being cut to just the Optimizer half.

5 · Safety & offline autonomy (your best moment)

3:15 – 4:15

"Here's the part I actually care most about. *[open Demo Scenarios, click "AI service failure"]* Watch the mode badge. *[point]* The AI just failed — on purpose, right now — and dispatch didn't stop. It fell back to a hard-coded safety controller instantly. That fallback sits outside the AI entirely; it cannot be overridden by it.

And connectivity — *[click "Internet outage"]* — the station just went offline. Telemetry, forecasting, optimization, safety: all of it keeps running at the edge. Only the cloud sync queues up, and when we reconnect *[toggle back online]* it catches up automatically. For a station where satellite links can vanish for weeks, that's not a nice-to-have — that's the entire point."

CUE Reset the scenario to "Normal operation" right after this segment, before moving on.

6 · Real engineering & impact

4:15 – 5:15

"This isn't only a frontend simulation. We built an independent Python backend — FastAPI, deployed live — running the exact same model as a real server, with a terminal client we can drive from a command line. *[optional: switch to VS Code, run cli.py, pick "Watch live status"]* I can trigger a change from a terminal, and it updates the live website in real time, because a real deployment would need exactly this: an API, a server, something a station's edge gateway could actually run.

On impact — *[click Green Audit tab]* — because our optimizer and the baseline run on identical conditions, this fuel-saved and CO₂-avoided number is a measured difference in our simulation, not an assumed one. We're not claiming this is what a real station would save; we're showing the method that would measure it honestly once it is."

CUE The VS Code/CLI beat is a bonus, not required — skip it if you're already tight on time.

7 · Close

5:15 – 5:45

"Diesel is expensive, connectivity isn't guaranteed, and failure has real consequences. POLAR-EMS is our proposed answer: forecast ahead, optimize continuously, and never let safety depend on the AI being right. It's a prototype today — but the edge-first design, the explainable dispatch, and the hard safety floor underneath it are exactly what a real deployment would need.

Thank you — happy to take questions, or show any part of this live again."

Before you walk up

On the laptop

1 Reset the simulation

Demo Scenarios → Normal operation, and hit Reset, so it opens clean.

2 Have the tab already open

polar-ems-web.vercel.app loaded and idle before you're called up — don't type the URL live.

3 Warm up the Render backend

If you're doing the CLI beat, hit /health once beforehand — free-tier cold start takes 30-50s and you don't want that live.

In your head

1 Know the 30-second version cold

This is your real insurance policy, not the full script.

2 Decide who says what, if two of you present

Common split: Presenter A does Segments 1, 2, 7 (problem, intro, close); Presenter B drives the live demo, Segments 3–6.

3 Skim the Judge Q&A doc once more

Especially "did you use AI coding tools" and "what AI is actually running" — those come up almost every time.

4 Zoom the browser to ~110%

Small text on a projector is the single most common demo failure that has nothing to do with your code.

If something breaks live

WIFI/PROJECTOR FAILS MID-DEMO

Say this out loud, don't panic-apologize: *"Actually — this is a good demonstration of the point. The dashboard itself needs zero internet to keep running; only the Live Backend panel does."* Then keep going on the main tabs, which need nothing.

RENDER BACKEND IS ASLEEP / CLI CAN'T CONNECT

Skip the CLI beat entirely — it was always marked optional. Nothing else in the script depends on it.

A JUDGE INTERRUPTS WITH A QUESTION MID-SEGMENT

Answer it in one or two sentences, then say *"let me actually show you that"* and continue — don't try to finish the interrupted sentence from before.

YOU'RE RUNNING LONG AND THERE'S A HARD CUTOFF

Cut in this order: second half of Segment 4 (Forecast) → the VS Code/CLI beat in Segment 6 → shorten Segment 1 to two sentences. Never cut Segment 5 — it's the strongest moment you have.